

Sign-Language Detection

Aryan Jain
ECE Department
Rutgers University - New Brunswick
aj879@scarletmail.rutgers.edu

Joshua Menezes
ECE Department
Rutgers University - New Brunswick
jam1092@scarletmail.rutgers.edu

Muralii Krishnan Thirumalai
ECE Department
Rutgers University - New Brunswick
mt1171@scarletmail.rutgers.edu

Abstract—This is a report on a Convolutional Neural Network based system for static sign-language letter detection using the Sign Language MNIST dataset and live webcam inference. The system consists of training, evaluation, and a live camera inference component. The model was trained using PyTorch and evaluated using a held-out test set. The results include confusion matrix visualizations and example predictions to qualitatively analyze per class performance and common confusions. This report documents dataset preparation, model architecture, training procedure, evaluation, and future enhancements.

Index Terms—Sign Language, Convolutional Neural Network, PyTorch, Computer Vision, Classification, Confusion Matrix

I. INTRODUCTION

Automatic recognition of sign language is an important technology for improving accessibility and communication between hearing-impaired communities and the general public. This work focuses on static alphabet signs, which are single-frame images representing letters, using the Sign Language MNIST dataset and a lightweight CNN model implemented in PyTorch. The system includes a training pipeline (train.py) responsible for fitting the model to the dataset, an evaluation script (test.py) that generates both quantitative metrics and visual diagnostic outputs, and a live webcam inference script (camera.py) that performs real-time classification of hand-sign images captured from a camera stream. This preliminary report documents the implementation choices, training setup, evaluation procedure, and initial analysis of the model’s behavior, offering a clear overview of how each component functions within the complete pipeline.

II. DATASET

The dataset used is the Sign Language MNIST-style CSV dataset consisting of grayscale 28x28 images stored as flattened rows. Each CSV row contains a class label followed by 784 pixel intensity values ranging from 0 to 255. Dataset preprocessing performed in the code involves reading the CSV file using pandas, reshaping each flattened pixel vector back into a 28x28 array while casting it to uint8 format, and applying a sequence of transformations before the data is fed into the network. These transformations include conversion to a PIL image, application of optional data augmentations, conversion to a tensor, and normalization to ensure consistent scaling across samples.

During training, the code applies several augmentations, including a random rotation of approximately 10 degrees as well as normalization of pixel values to have a mean of 0.5

and a standard deviation of 0.5. Data loading is implemented via a PyTorch Dataset class called `SignLanguageMNIST`, combined with a `DataLoader` that uses a batch size of 64 and shuffles the data during training to improve generalization by preventing the model from memorizing sample order.

III. MODEL ARCHITECTURE

The model is a compact CNN designed for 28x28 grayscale inputs. Its architecture includes an initial convolutional layer (`Conv2d(1, 32, kernel=3)`) followed by a ReLU activation function and a max-pooling operation with a 2x2 window. A second convolutional block (`Conv2d(32, 64, kernel=3)`) is applied next, again followed by ReLU activation and a second max-pooling layer. After these convolutional stages, a dropout layer with probability 0.25 is used to reduce overfitting by randomly dropping units during training. The classifier portion consists of a fully connected layer mapping the flattened feature map ($64 \times 5 \times 5$) to 256 units, followed by another ReLU activation and dropout, and then a final fully connected layer projecting the 256-dimensional vector to the number of output classes. The model uses `CrossEntropyLoss` as the final loss function.

It is important to note that the final linear layer in the training code outputs 25 logits. Therefore, users must ensure that the mapping between class indices and semantic labels (A to Z with dynamic letters removed) remains consistent across the dataset, training labels, and the camera-inference script. The camera script builds a list of alphabet labels from A to Z but removes the letters J and Z; hence, verifying perfect alignment between this mapping and the class indices used during training and evaluation is essential to avoid unnecessary mismatches.

IV. TRAINING SETUP

Training hyperparameters and settings as implemented include the use of the Adam optimizer, a learning rate of 0.001, `CrossEntropyLoss` as the objective function, a batch size of 64, a total of 10 training epochs, and a dropout rate of 0.25. The model is trained on a CUDA-capable GPU when one is available; otherwise, it defaults to the CPU without changing the logic of the training loop.

The training loop computes the running loss for each epoch and tracks training accuracy by comparing the number of correct predictions to the total number of samples in the training set. Once training concludes, the final model state

dictionary is saved under the filename `sign_model.pt`, making it available for evaluation and real-time inference.

V. EVALUATION PROCEDURE

Evaluation is implemented in `test.py`. The evaluation process begins by loading the saved model `sign_model.pt` and switching it into evaluation mode. Next, the script iterates through the test `DataLoader` using batches of size 64, collecting predictions and corresponding labels across the entire test set. The script then calculates the overall test accuracy and prints it to the console to provide a simple quantitative metric. Afterward, it computes and visualizes the confusion matrix using `scikit-learn`'s `confusion_matrix` and `ConfusionMatrixDisplay`, saving the result to `outputs/confusion_matrix.png`. Finally, a grid of twelve randomly selected test images, along with their predicted and true labels, is generated and saved as `outputs/sample_predictions.png` to enable qualitative inspection of model behavior.

VI. RESULTS

A. Quantitative Metric

The evaluation script reports a single quantitative metric, namely the test accuracy. When running the testing code `test.py` we got the test accuracy to be **95.87%**.

B. Confusion Matrix

Figure 1 shows the confusion matrix computed over the test dataset, saved in `outputs/confusion_matrix.png`. The confusion matrix enables examination of per-class performance and highlights common misclassifications by revealing which letters the model tends to confuse with one another. These insights help identify classes that might require additional data collection or targeted augmentation strategies to improve differentiation between visually similar signs.

C. Qualitative Predictions

Figure 2 presents a set of twelve randomly sampled test images, each annotated with both their true label and the model's predicted label. This visualization, saved to `outputs/sample_predictions.png`, helps verify that the model correctly captures key hand-shape characteristics for many letters. It also highlights ambiguous cases such as those containing occlusion, motion blur, or unusual hand positioning, where these visual issues reduce image clarity and increase the likelihood of misclassification by obscuring essential hand features.

VII. ANALYSIS AND DISCUSSION

The confusion matrix and sample predictions reveal several notable observations. First, most classes are recognized reliably, as seen in the strong diagonal pattern within the confusion matrix that indicates a high number of correctly identified letters. Second, some classes exhibit higher off-diagonal counts, meaning the model frequently confuses them with similar-looking letters, particularly when hand shapes are

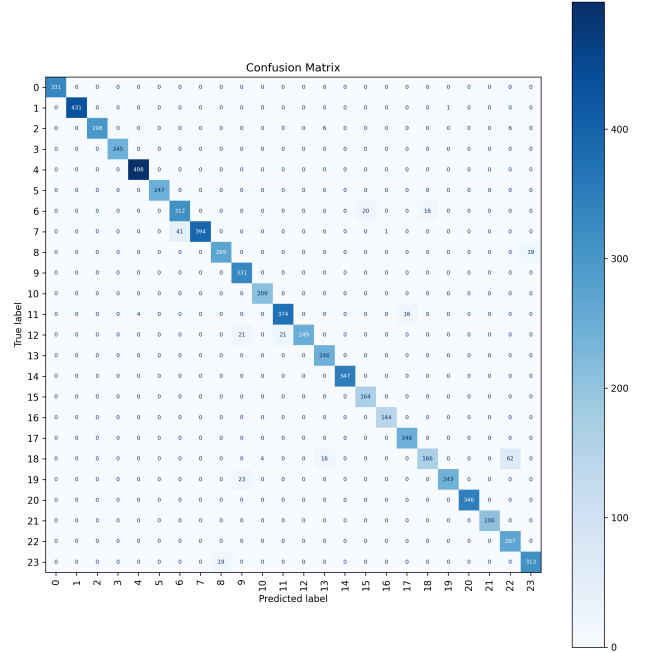


Fig. 1: Confusion matrix across all evaluated classes (saved as `outputs/confusion_matrix.png`). Diagonal values are per-class true positives, and off-diagonals indicate confusions that can provide insight into which letters are visually similar.

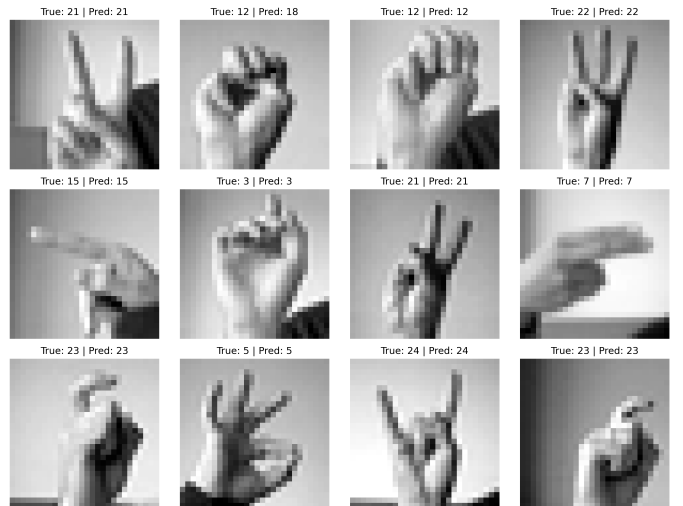


Fig. 2: Random sample predictions (12 images) from the test set with 'True' and 'Pred' labels shown in the subcaptions. Saved as `outputs/sample_predictions.png`.

visually similar or the image exhibits low contrast or partial occlusion. Third, occasional misclassification may arise from inconsistencies in preprocessing or mismatches between the dataset's label ordering and the model's output indexing, making it important to verify that label mappings are consistent throughout training, evaluation, and camera-based inference. Finally, although the model is relatively small and trains efficiently, its limited architectural complexity may hinder performance on more challenging distinctions, suggesting that additional model capacity could improve recognition fidelity.



Fig. 3: This is the output of our model (Top Picture shows A and the Bottom Picture Shows Y).

VIII. LIMITATIONS AND POTENTIAL IMPROVEMENTS

We recommend several improvements for achieving a more robust and accurate system. First, label consistency must be ensured so that the same mapping from class indices to alphabet letters is used across training, testing, and live camera inference, particularly because inconsistencies can lead to misinterpretations. Second, introducing a validation split enables monitoring of validation accuracy and selecting

the best-performing model checkpoint rather than merely saving the final epoch. Third, expanding data augmentation with methods such as translations, brightness and contrast adjustments, subtle affine transformations, and random erasing would likely improve robustness against environmental variability by exposing the model to a broader range of input conditions. Fourth, the architecture itself could be improved by adding Batch Normalization, increasing the number of filters or convolutional layers, or transitioning to modern lightweight architectures such as MobileNet or EfficientNet-lite to achieve higher accuracy without sacrificing inference speed. Fifth, enhancing the live camera pipeline by incorporating a dedicated hand detector such as MediaPipe Hands would allow automatic region-of-interest cropping, improving real-time performance and reducing background interference. Sixth, applying temporal smoothing such as majority voting or exponential moving averages across sequential camera frames would help reduce classification flicker during live use. Seventh, computing per-class metrics such as precision, recall, and F1-score using tools like scikit-learn's `classification_report` would help diagnose specific classes that require additional augmentation or relabeling. Finally, ensuring reproducibility by setting random seeds for NumPy, PyTorch, and Python's `random` library, as well as logging hardware, library versions, and other metadata, would help make experiments more consistent and easier to replicate.

IX. CONCLUSIONS

This preliminary study demonstrates a working pipeline for static sign-language alphabet recognition using a compact PyTorch CNN. The training and testing scripts successfully produce a saved model and visual evaluation artifacts, such as the confusion matrix and sampled predictions. The current model provides a strong starting point, and targeted dataset improvements, architecture enhancements, and better camera region-of-interest strategies will further raise performance and robustness for real-world use.

ACKNOWLEDGMENT

We thank each other for the team effort on building the training, evaluation, and live inference pipeline. We also thank the public datasets and the PyTorch community for the tooling used in this project.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Proc. NIPS, 2012.
- [2] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in NeurIPS, 2019.
- [3] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, 1998.
- [4] T. Baltrušaitis, C. Ahuja and L.-P. Morency, "Multimodal Machine Learning: A Survey and Taxonomy," Foundations and Trends in ML, 2019.
- [5] Kaggle, "Sign Language MNIST," <https://www.kaggle.com/competitions/sign-language-mnist>, accessed 2025.
- [6] M. K. Hasan, S. Igdeg, "A Short Survey on Sign Language Recognition," Journal of Accessibility Research, 2020.
- [7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," 2014.